Sorting Algorithms

Sort Colors / Dutch National Flag problem

Core Idea

Given an array containing only `0s`, `1s`, and `2s`, sort it **in-place in a single pass** without using any built-in sort. This is also known as the **Dutch National Flag problem** proposed by Edsger Dijkstra. The idea is to maintain three pointers that partition the array into three sections simultaneously.

The Three Pointers

- **low** — everything before low is `0` (red)
- **mid** — current element being examined
- **high** — everything after high is `2` (blue)
- Everything between low and high is `1` (white)

```
[0...0 | 1...1 | unsorted | 2...2]
         ↑         ↑           ↑
        low       mid         high
```

Steps

1. Start with `low = 0`, `mid = 0`, `high = end of array`
2. Look at the element at `mid`:
   - If `nums[mid] == 0` → swap with `low`, move both `low` and `mid` forward
   - If `nums[mid] == 1` → it's in the right place, just move `mid` forward

- If `nums[mid] == 2` → swap with `high`, move `high` backward (**don't move mid**)

3. Repeat until `mid > high`

Visualizing It

`[2, 0, 1, 2, 0]`

low = 0, mid = 0, high = 4

Step 1: `nums[mid] = 2` → swap mid and high → move high back

```
[0, 0, 1, 2, 2]  low=0, mid=0, high=3
```

Step 2: `nums[mid] = 0` → swap mid and low → move both forward

```
[0, 0, 1, 2, 2]  low=1, mid=1, high=3
```

Step 3: `nums[mid] = 0` → swap mid and low → move both forward

```
[0, 0, 1, 2, 2]  low=2, mid=2, high=3
```

Step 4: `nums[mid] = 1` → just move mid forward

```
[0, 0, 1, 2, 2]  low=2, mid=3, high=3
```

Step 5: `nums[mid] = 2` → swap mid and high → move high back

💡 `[0, 0, 1, 2, 2]  low=2, mid=3, high=2`

mid > high → **stop** ✅

```
Final: [0, 0, 1, 2, 2] ✅
```

Key Characteristics

- **Not Stable** — swapping disturbs original order
- **In-place** — no extra array needed
- **Single pass** — mid only moves forward, each element is touched at most once

Time & Space Complexity

Case	Time	Why
Best	O(n)	Single pass through the array
Average	O(n)	Single pass always
Worst	O(n)	Single pass always

Space: O(1)

Why mid Doesn't Move When Swapping with high

When you swap `mid` with `high`, the element that comes to `mid` is **unknown and unexamined yet** — it just arrived from the unsorted region. So you must examine it again before moving forward. But when you swap `mid` with `low`, you know the element coming from `low` is a `1` (already processed), so it's safe to move `mid` forward.

Why It's Asked in Interviews

It's a clean and elegant application of the **two/three pointer technique**. Interviewers use it to test whether you can think beyond sorting algorithms and solve a constrained problem optimally in a single pass.